No other output is permitted. In both compilers, the root cause is insufficiently conservative treatment of a pointer derived from an integer. The problem is difficult to solve without throwing away desirable pointer optimizations. This program has been reported in the bug tracking systems for both compilers. The semantics given in this paper solve this problem, and we have confirmed that our prototype fixes this bug.

B APPENDIX: SAFE RUST PROGRAM MISCOMPILED BY LLVM

This function uses low-level language features, but it is still in the safe subset of Rust. It is miscompiled because of a bug in LLVM's GVN optimization:

```
pub fn test(gp1: &mut usize, gp2: &mut usize, b1: bool, b2: bool) -> (i32, i32) {
    let mut g = 0;
    let mut c = 0;
    let y = 0;
    let mut x = 7777;
    let mut p = &mut g as *const _;

    {
        let mut q = &mut g;
        let mut r = &mut 8888;

        if b1 {
            p = (&y as *const _).wrapping_offset(1);
        }
        if b2 {
            q = &mut x;
        }

        *gp1 = p as usize + 1234;
        if q as *const _ == p {
            c = 1;
            *gp2 = (q as *const _) as usize + 1234;
            r = q;
        }
        *r = 42;
    }
    return (c, x);
}
```

This function first assigns a reference of `g` to `q`. It then creates a temporary object holding the number `8888` and `r` is assigned a reference to it. Function `wrapping_offset` performs pointer arithmetic that can safely go out of bounds of the base object (equivalent to LLVM's **gep** without **inbounds**). If `b2` is true, `q` is assigned a reference to `x`. Therefore, if the program enters in the following branch as well, `r` is assigned a reference to `x`, and thus the following store through `r` (`*r = 42;`) overwrites the value of `x`.

When called with `b1` and `b2` set to true, the optimized version of this code returns `c = 1, x = 7777`. This outcome is impossible; legal results are `c = 0, x = 7777` and `c = 1, x = 42`.

The miscompilation happens when LLVM's GVN pass exploits the condition of the last `if` statement and incorrectly replaces all uses of `q` within that branch with `p`. After the replacement, `*r` is assumed not to touch `x` because `r` now either contains a reference to the initial temporary object or to one of the references assigned to `p` (based on `g` and `y` only). Therefore, `x = 7777` is constant-propagated to the return statement, which causes the wrong output.